

FPGA UART Command Controller

Explained Simply — what it is, how it works, and what was tested

What Is This Project?

This is a small digital circuit, written in a hardware description language called **VHDL**, that you could run on an FPGA (a reprogrammable chip). It does one simple job: you type characters into a computer, those characters travel over a serial connection called **UART**, and the chip uses them to control how bright 4 separate LEDs glow.

Think of it like a tiny remote control: instead of buttons, you type letters and numbers, and instead of a TV, it's controlling LED brightness.

How It Works (3 Simple Stages)

1. Listen	2. Decide	3. Glow
The UART Receiver listens to the serial line and catches one typed character at a time.	The Command Decoder looks at the character and decides: "which LED?" and "how bright?"	The PWM Generator turns that decision into a flickering signal fast enough to look like steady brightness to your eyes.

PWM stands for **Pulse Width Modulation** — it's a trick where you flip an LED on and off very fast. If it's on most of the time, it looks bright. If it's on only a little of the time, it looks dim. There's no analog dimmer here, just fast digital switching.

The Command Language

There are two kinds of characters you can type. One kind picks *which* LED you're talking to, and the other kind sets *how bright* it should be.

Character	What it does
A, B, C, D	Select which LED (1st, 2nd, 3rd, or 4th) you want to control next
0 – 9	Set the brightness of whichever LED is currently selected (0 = off, 9 = full brightness, 10 steps total)
Anything else	Ignored — typos don't break anything
Reset button	All 4 LEDs turn off and selection resets to LED #1

Example: typing B5 selects the 2nd LED, then sets it to about halfway brightness. Typing just 9 by itself (no letter first) affects whichever LED was selected most recently.

What's in the Project (File by File)

File	In plain English
uart_led_pkg.vhd	A shared settings file: says there are 4 LEDs, and defines a reusable "list of 4 brightness values" data type.
uart_rx.vhd	The 'ears' of the system. Watches the serial wire bit by bit and reassembles each typed character.
command_decoder.vhd	The 'brain'. Remembers which LED is selected and what brightness each of the 4 LEDs should be.
pwm_generator.vhd	The 'muscle'. One copy runs per LED, turning a brightness number into the actual on/off flicker pattern.
uart_led_top.vhd	Wires the 3 pieces above together into one complete design.
tb_uart_led_top.vhd	The test script. Pretends to type commands and checks that the LEDs respond correctly — no human needed to watch it.
arty_a7.xdc	Tells real FPGA hardware (a Digilent Arty A7 board) which physical pins the clock, reset, UART, and LEDs connect to.

How We Know It Actually Works

Instead of guessing, the project includes an automatic test script that acts like a robot typist: it sends commands to the design and checks the LED brightness after each one, the same way a human would check with a multimeter — but instantly and repeatably.

The test script checked 7 scenarios (14 individual checks total):

1. Right after power-on / reset, all 4 LEDs are off
2. Sending '5' with no LED selected yet still controls LED #1 by default
3. Selecting LED #2 and setting it full brightness doesn't disturb LED #1
4. Selecting LED #3 and turning it off works correctly
5. Typing an unsupported character (like 'Z') is safely ignored
6. Hitting reset mid-operation turns everything back off
7. After that reset, the system still works correctly (picks back up with LED #4)

This was run for free in a browser using a site called **EDA Playground**, with a simulator called **GHDL** — no FPGA board or paid software needed to prove the logic works.

```

Log  Share
[2026-10-01 23:44:23 UTC] ghd -i pwm_generator.vhd uart_rx.vhd command_decoder.vhd uart_led_pkg.vhd design.vhd testbench.vhd && ghd -m tb_uart_led_top && ghd -r tb_uart_led_top
analyze design.vhd
analyze uart_rx.vhd
analyze command_decoder.vhd
analyze pwm_generator.vhd
analyze uart_led_pkg.vhd
analyze testbench.vhd
elaborate tb_uart_led_top
testbench.vhd:75:13:@26350us:(report note): TEST 1 (LED0 off after reset) PASS (duty=0%)
testbench.vhd:75:13:@51950us:(report note): TEST 1 (LED1 off after reset) PASS (duty=0%)
testbench.vhd:75:13:@77550us:(report note): TEST 1 (LED2 off after reset) PASS (duty=0%)
testbench.vhd:75:13:@103150us:(report note): TEST 1 (LED3 off after reset) PASS (duty=0%)
testbench.vhd:75:13:@138950us:(report note): TEST 2 (LED0 mid brightness from 'S', no channel select needed) PASS (duty=55%)
testbench.vhd:75:13:@184950us:(report note): TEST 3 (LED1 full brightness from 'B'+9') PASS (duty=99%)
testbench.vhd:75:13:@210550us:(report note): TEST 3 (LED0 unaffected by channel switch) PASS (duty=55%)
testbench.vhd:75:13:@256550us:(report note): TEST 4 (LED2 off from 'C'+0') PASS (duty=0%)
testbench.vhd:75:13:@292350us:(report note): TEST 5 (LED1 unchanged after unsupported 'Z') PASS (duty=99%)
testbench.vhd:75:13:@318750us:(report note): TEST 6 (LED0 off after mid-op reset) PASS (duty=0%)
testbench.vhd:75:13:@344350us:(report note): TEST 6 (LED1 off after mid-op reset) PASS (duty=0%)
testbench.vhd:75:13:@369950us:(report note): TEST 6 (LED2 off after mid-op reset) PASS (duty=0%)
testbench.vhd:75:13:@395550us:(report note): TEST 6 (LED3 off after mid-op reset) PASS (duty=0%)
testbench.vhd:75:13:@441550us:(report note): TEST 7 (LED3 low-mid brightness from 'D'+3' after reset) PASS (duty=33%)
testbench.vhd:196:9:@441550001ns:(report note): =====
testbench.vhd:197:9:@441550001ns:(report note): TESTS PASSED: 14
testbench.vhd:198:9:@441550001ns:(report note): TESTS FAILED: 0
testbench.vhd:200:13:@441550001ns:(report note): ALL TESTS PASSED
testbench.vhd:204:9:@441550001ns:(report note): =====
Finding VCD file...
./dump.vcd
[2026-10-01 23:44:25 UTC] Opening EPWave...
Done
```

Actual output from the test run: every single check says PASS, and the summary confirms 14 passed, 0 failed.

Current Status

Verified in simulation only. The logic has been proven correct inside a simulator, but it has not yet been loaded onto a real FPGA board. That's an honest and normal stopping point for a portfolio project — the pin-mapping file (arty_a7.xdc) is already prepared for a Digilent Arty A7 board if you want to take that next step later.

Where Everything Lives

All the source code, the test script, the pin-mapping file, and this explanation are saved to GitHub at:

github.com/muraliikrishnant/UART_FPGA

One-Sentence Summary

"I designed a small FPGA system where typed commands over a serial connection control the brightness of 4 independent LEDs, and I proved it works with an automated test suite covering 7 real-world scenarios."